PDF

Lab_4_VibeCoding_Student_handout (1)(2).pdf
PDF

I have a Software Engineering lab deliverable based on a Coin Collector Pygame project.

I have attached the project README and the lab deliverable/instructions. Please read them first and do not ma
changes yet.

I will give you the tasks one by one. For each task, implement only that task, preserve the previous functionalit
make the smallest clean changes necessary. After each task, explain which files were modified and why.

Coin Collector Lab

This project is a single-topic top-down Coin Collector game using **Pygame**. It introduces students to per-frame
collision bookkeeping, entity variety, an obstacle/lives system, and round timing, using a small, readable object
oriented codebase.

What's Provided

A working Coin Collector game with:

- A player that moves around a play area with the arrow keys
- Coins scattered around the play area that award points on contact
- A running score display

It has **one deliberate bug** and **three features** left for you to build. You are expected to **analyze**, **interact with
assistant**, and **complete/fix** the game to make it fully functional and more interesting.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programmi
partner for this lab.

Getting Started

Setup

1. Make sure you have Python 3.10+ installed.
2. Install dependencies:

```
pip install -r requirements.txt
```

3. Run the game:

```
python main.py
```

Controls: Arrow keys to move.

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code revie

Task 1: Fix the coin-collection bug

A coin is supposed to be collected exactly once, the moment the player touches it. In the current build, `update()` (in `game/game_engine.py`) calls `check_collection` every frame and adds a coin's value to the score for as long as the player's rectangle keeps overlapping it - but the coin is never actually removed after being collected. Just walking through a single coin at normal speed (without stopping) scores it more than a dozen times in one pass. Fix it so each coin is collected exactly once, no matter how long the player stands on or walks through it.

Task 2: Implement multiple coin types

Introduce at least three coin types with different values - for example bronze (1 point), silver (3 points), and gold (5 points). Give each type its own color so they're visually distinguishable, and make sure the correct value is awarded when each type is collected.

Task 3: Implement obstacles

Add obstacles to the play area that the player must avoid while collecting coins. Colliding with an obstacle should have a clearly defined consequence (for example, losing a life). Obstacles should stay within the play area and interact correctly with the player.

Task 4: Implement a timed round

Add a 30-second countdown for the round. Display the remaining time on screen. Once it reaches zero (or lives run out, once Task 3 is done), stop the round, show the final score clearly, and provide a way to start a new round with the score, lives, and timer all reset.

Expected Behavior

- Walking through or standing on a coin should collect it exactly once - the score should not keep climbing the time the player happens to be touching it.
- Coin types are visually distinguishable and award the correct value.
- Touching an obstacle has a real, clearly defined consequence, but a single touch shouldn't repeatedly punis player every frame they're still overlapping it.
- The round ends when time runs out or lives reach zero, whichever comes first, with the final score shown cl and a way to start again.

Folder Structure

Folder Structure

```
coin-collector/
├── main.py
├── requirements.txt
├── game/
│   ├── game_engine.py
│   ├── player.py
│   ├── coin.py
│   ├── collection.py
│   └── renderer.py
└── README.md
```

Submission Checklist

Submission is only the following three things:

- A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- The Chat/LLM used page link, with the complete chat history

Understood. I've read the lab handout and the README content you provided. The handout confirms that the fea must be added **one by one**, with prompts/chat history retained, and that the final submission includes before/afte updated code, and exported chat history.

I won't make any changes yet.

Send **Task 1** when you're ready.

Fix Task 1 from the README only. A coin must be collected exactly once when the player touches it. Currently, `update()` adds the coin's value every frame while the player overlaps the coin because the collected coin is ne removed. Fix this behavior so the coin is removed after collection and cannot award points again. Preserve all existing functionality, and do not implement Tasks 2, 3, or 4. Make the smallest clean change necessary and ex which file you modified and why.

I can implement Task 1, but the actual project source files are **not attached/available in this chat**. I only have th handout, which confirms the task workflow but does not contain `game_engine.py` or the other project files.

Please attach the **Coin Collector project/repository files** (or at minimum `game/game_engine.py` and `game/collection.py`/`coin.py` if applicable).

Once they're available, I'll make **only the Task 1 change**, with no Tasks 2–4, and report exactly which file(s) were modified and why.

""" GameEngine: owns the player and all coins. Starter version: one coin type, no obstacles, no timer yet. Coin collection also has a known bug (see how `update` uses check_collection below) that Task 1 asks you to fix - coins are never removed, so standing on one keeps awarding points every frame. """ import random import pyg from game.player import Player from game.coin import Coin from game.collection import check_collection from game.renderer import WIDTH, HEIGHT NUM_COINS = 6 COIN_VALUE = 1 class GameEngine: def __init__(s self.player = Player(x=WIDTH / 2, y=HEIGHT / 2) self.coins = [self._random_coin() for _ in range(NUM_COINS self.score = 0 def _random_coin(self): x = random.randint(30, WIDTH - 30) y = random.randint(30, HEIGHT - 3 return Coin(x=x, y=y, radius=12, value=COIN_VALUE) def handle_input(self, keys_pressed): dx = dy = 0 if keys_pressed[pygame.K_UP]: dy -= self.player.speed if keys_pressed[pygame.K_DOWN]: dy += self.player.sp keys_pressed[pygame.K_LEFT]: dx -= self.player.speed if keys_pressed[pygame.K_RIGHT]: dx += self.player. self.player.move(dx, dy, WIDTH, HEIGHT) def update(self): collected = check_collection(self.player, self.coins) coin in collected: self.score += coin.value def draw(self, surface, font): from game import renderer renderer.draw_scene(surface, self.player, self.coins) renderer.draw_text(surface, font, f"Score: {self.score}", (1

Yes — **Task 3 is done** based on the implementation above.

You now have:

- ✅ Obstacles inside the play area
- ✅ 3 lives
- ✅ Collision detection
- ✅ One-life consequence per collision
- ✅ Player reset after collision so the same overlap doesn't repeatedly deduct lives
- ✅ Task 1 preserved
- ✅ Task 2 preserved
- ❌ Task 4 not implemented

One important thing: **make sure you actually apply the Task 3 code before committing**. Then you can test obs collisions and lives decrementing.

Implement Task 4 from the README only: add a 30-second countdown timer to the round and display the rema time on screen. The round must end when the timer reaches zero or when the player loses all lives, and the fin score should be displayed. Provide a way to restart the round after it ends, with the score, lives, timer, and gam state reset appropriately. Preserve all existing Task 1, Task 2, and Task 3 behavior. Do not make any changes b what is necessary for Task 4. Explain which files were modified and why.

Please paste the current **`main.py`**.

Task 4 needs the restart action (e.g. pressing `R`) to be handled by the existing game loop, so I need that file to ma
smallest possible change and avoid altering anything from Tasks 1–3.

""" Coin Collector (Lab Starter) Run with: python3 main.py Controls: Arrow keys to move. """ import pygame fro
game.game_engine import GameEngine from game.renderer import WINDOW_SIZE def main(): pygame.init()
screen = pygame.display.set_mode(WINDOW_SIZE) pygame.display.set_caption("Coin Collector") clock =
pygame.time.Clock() font = pygame.font.SysFont("consolas", 22) engine = GameEngine() running = True while
running: for event in pygame.event.get(): if event.type == pygame.QUIT: running = False keys =
pygame.key.get_pressed() engine.handle_input(keys) engine.update() engine.draw(screen, font)
pygame.display.flip() clock.tick(60) pygame.quit() if __name__ == "__main__": main()

Yes. With the `main.py` you provided, Task 4 can be implemented with changes to **`game/game_engine.py`**,
`game/renderer.py`, and **`main.py`**.

1. `game/game_engine.py`

Add the Task 4 constants:

```
ROUND_DURATION = 30
```

In `__init__()`, add:

```
self.round_active = True
self.start_time = pygame.time.get_ticks()
self.time_remaining = ROUND_DURATION
```

At the beginning of `handle_input()`:

```
if not self.round_active:
    return
```

Update `update()` to:

```
def update(self):
    if not self.round_active:
        return

    elapsed = (pygame.time.get_ticks() - self.start_time) // 1000
    self.time_remaining = max(0, ROUND_DURATION - elapsed)

    if self.time_remaining == 0 or self.lives == 0:
        self.round_active = False
        return

    collected = check_collection(self.player, self.coins)

    for coin in collected:
        self.score += coin.value
        self.coins.remove(coin)

    for obstacle in self.obstacles:
        if self.player.get_rect().colliderect(obstacle.rect):
            self.lives = max(0, self.lives - 1)
            self.player.x = WIDTH / 2
            self.player.y = HEIGHT / 2
            break
```

Update `draw()`:

```
def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface, self.player, self.coins, self.obstacles
    )

    renderer.draw_text(
        surface,
        font,
        f"Score: {self.score}   Lives: {self.lives}   Time: {self.time_remaining}",
        (10, 10),
    )

    if not self.round_active:
```

```
            renderer.draw_banner(
                surface,
                font,
                f"Round Over! Final Score: {self.score} - Press R to restart",
            )
```

Finally, add:

```
def reset_round(self):
    self.player = Player(x=WIDTH / 2, y=HEIGHT / 2)

    self.coins = [
        self._random_coin(COIN_TYPES[i % len(COIN_TYPES)])
        for i in range(NUM_COINS)
    ]

    self.lives = NUM_LIVES
    self.score = 0
    self.time_remaining = ROUND_DURATION
    self.start_time = pygame.time.get_ticks()
    self.round_active = True
```

2. `main.py`

Add handling for the `R` key:

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False
    elif event.type == pygame.KEYDOWN:
        if event.key == pygame.K_r and not engine.round_active:
            engine.reset_round()
```

Everything else in `main.py` remains unchanged.

3. `game/renderer.py`

No structural change is required if you already have `draw_banner()` from Task 3. The existing function can displa
final-score/restart message.

Result

Task 4 now provides:

- ✅ 30-second countdown
- ✅ Remaining time displayed
- ✅ Round ends at `0` seconds
- ✅ Round ends when lives reach `0`
- ✅ Final score displayed
- ✅ `R` restarts after the round ends
- ✅ Score resets
- ✅ Lives reset to 3
- ✅ Timer resets to 30
- ✅ Player/coins reset
- ✅ Game state becomes active again
- ✅ Task 1 preserved
- ✅ Task 2 preserved
- ✅ Task 3 preserved

Files modified:

- `game/game_engine.py` — timer, round state, reset logic, countdown display
- `main.py` — handles `R` to restart after game over
- `game/renderer.py` — **no changes necessary** because the existing banner already supports the required fin message.